

Mary Ann Mastri
Computer Vision
6263-14050-ITAI-1378
June 28, 2026

Reflective Journal for Lab 05: Chihuahua vs Muffin Classifier using Convolutional Neural Networks

1. CNN Architecture

- Briefly describe the CNN architecture and how it differs from the traditional neural network used in the previous workshop.

CNNs use convolutional layers, filters, pooling layers and then fully connected layers. Because filters are shared across the image, CNNs have a lower parameter count, learn feature hierarchy and preserves spatial awareness. This structure allows CNNs to detect patterns like edges, textures, and shapes in ways that traditional neural networks cannot.

Traditional neural networks only use fully connected layers where every neuron connects to every neuron in the next layer. This leads to a very high parameter count, no built-in spatial awareness and require manual feature engineering. (Goodfellow, Bengio, & Courville, 2016).

2. Model Performance

- Observe and report on the model's performance, including accuracy and any interesting patterns in the misclassifications.

I spent most of my time on hyperparameter tuning, and it also frustrated me the most.

Dropout values

I experimented with dropout values of 0.5, 0.7 and 0.8. Each time I changed a dropout, I had to retrain the model from scratch, which took forever on a CPU. The results were not necessarily worth the wait, but did prove the following:

- Lower dropout: higher training accuracy but more overfitting
- Higher dropout: lower training accuracy and better generalization
- Dropout = 0.8 made the model work extra hard to learn anything, but it also reduced overfitting the most

Epochs

I increased epochs from 10 to 15 to 20. The model improved up to around 15 epochs, then it leveled off. More wasn't always better, it just took way more time.

The Plotting Disaster

I added code to track losses, ran training and nothing plotted. I checked everything. I rewrote code, stared at the monitor, then realized I forgot to call:

```
(Python) plot_loss_curves(train_losses, val_losses)
```

I spent way too much time on this but once identified and fixed, the plots finally appeared and gave me more intuitive visual understanding.

Added Visuals

I added:

- Accuracy curves
- Confusion matrix
- Per-class accuracy
- Misclassified image viewer

Seeing the misclassified images made me feel better. At least it made me laugh.

If I had more time and a GPU, I would:

- Experiment with different optimizers
- Add more convolutional layers

- Increase the image size
- Add far more data samples!

The dataset was tiny so even small changes caused the accuracy to swing wildly.

3. Comparison

- Compare the CNN with the traditional neural network model in terms of performance and training time.

CNNs are usually able to train faster and perform better because they share filters and local receptive fields which reduces the number of parameters and it allows them to learn spatial features quickly. CNNs generalize better because they learn spatial hierarchies automatically.

Traditional Neural Networks are slower to train and perform worse on image tasks because they rely on fully connected layers with very large parameter counts and do not have built in spatial awareness. (Goodfellow, Bengio, & Courville, 2016).

4. Challenges and Solutions

- Reflect on the challenges you faced during the lab and how you overcame them.

The data set was small so I easily achieved overfitting! I realized that anytime I changed the architecture or parameters, I needed to rerun the model from the beginning to avoid using overtrained weights.

To reduce overfitting, I used dropout, which randomly disables neurons during training as well as early stopping. I trained for 50 epochs and all the model did was memorize everything (and eat up a lot of time), so I just reduced the training to 15 epochs.

Realistically, I would need thousands of images and not just 30. Those images should be diverse and include a variety of images with:

- unusual angles
- blurry images
- partial images
- low and bright lighting
- continuously train with new images (Shorten & Khoshgoftaar, 2019)

5. Real-World Applications:

- Consider potential real-world applications of this type of image classification model.
 - Medical Imaging – Models classify X-rays, MRIs, ultrasounds to help identify disease.
 - Security and surveillance – detecting objects or people that do not belong in restricted areas.
 - Retail automation – Image classification can track inventory and also shelf-stock monitoring.
 - Ag monitoring – Farmers can use drone images to identify plant diseases, crop maturity, or pests/infestation.
 - Robotics – Use image classification to recognize objects, move around warehouses, and make decisions in real time.

6. Ethical Considerations:

- Discuss any ethical considerations regarding the development and deployment of such models.

Even though this lab was not of ethical concern with dogs and muffins, it still touches on real ethical issues:

- People might trust the model more than they should (medical imaging)
- Small datasets introduce bias
- Image data must be handled responsibly and strip PII and be secured with strict access controls.
- The models can misclassify images in harmful ways.
- Users should know the model's limitations and what it is trained to do. (Mitchell et al., 2019).

References

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1), 1–48.

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., ... & Gebru, T. (2019). Model cards for model reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency* (pp. 220-229). ACM.